SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	GIFTSON KEVIN MATTHEW.Y	**Reg.No.:**	192511167
Department:	B.Tech (CSE)	**Date:**	7/8/2026

Code of Conduct:

I GIFTSON KEVIN MATTHEW.Y (192511167)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIFTSON KEVIN MATTHEW.Y

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment No: 1

Title

Design and Simulation of Register Transfer Operations Using Arithmetic Logic Unit (ALU)

Aim

To design and simulate a register transfer system that performs arithmetic operations (Addition and Subtraction) and logical operations (AND and OR) using multiple registers and an Arithmetic Logic Unit (ALU), and to analyze the sequence of data transfer between registers during execution.

Objective

The objectives of this experiment are:

- To understand the concept of Register Transfer Language (RTL).
- To study the organization of processor registers.
- To perform arithmetic operations using an ALU.
- To perform logical operations using an ALU.
- To observe data movement between registers.
- To analyze the role of the ALU in executing processor instructions.
- To understand how registers cooperate during instruction execution.

Theory

Register Transfer

A **register** is a small, high-speed storage element inside the CPU that temporarily stores data, instructions, or addresses during program execution.

Register Transfer refers to the movement of binary information from one register to another under the control of clock signals.

The transfer is represented using **Register Transfer Language (RTL).**

Example:

R2 ← R1

This means that the contents of register **R1** are copied into **R2**.

Register transfer is one of the most fundamental operations performed inside every processor. Before any computation is carried out, data must first be transferred into appropriate registers.

Register Transfer Language (RTL)

RTL is a symbolic notation used to describe operations occurring between registers.

Examples:

R1 ← R2

R3 ← R1 + R2

R4 ← R2 AND R3

R5 ← R3 OR R4

RTL provides a clear representation of how data flows inside the processor.

Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the computational core of the processor. It performs all arithmetic and logical operations.

The ALU receives inputs from processor registers, performs the specified operation, and stores the result back into another register.

The ALU typically performs operations such as:

- Addition
- Subtraction
- Increment
- Decrement
- AND
- OR
- XOR
- NOT
- Shift operations
- Comparison operations

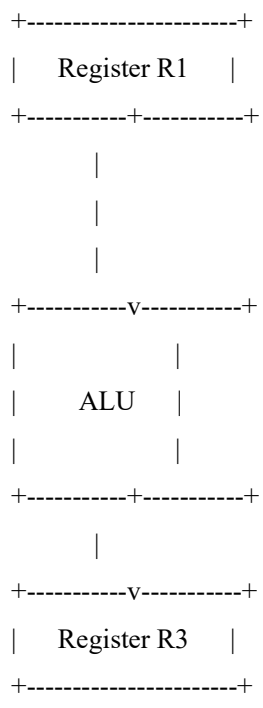
Without the ALU, the CPU would only be capable of storing data and would not be able to perform computations.

Registers Used

For this experiment, four general-purpose registers are used.

Register	Initial Value (Decimal)	Binary Representation
R1	25	00011001
R2	15	00001111
R3	0	00000000
R4	0	00000000

Block Diagram



The registers provide inputs to the ALU. The ALU performs the selected operation, and the result is stored in the destination register.

Algorithm

Step 1

Initialize all processor registers.

Step 2

Load data into R1 and R2.

Step 3

Transfer R1 into R3.

Step 4

Perform Addition.

R3 ← R1 + R2

Step 5

Perform Subtraction.

R4 ← R1 − R2

Step 6

Perform Logical AND.

R3 ← R1 AND R2

Step 7

Perform Logical OR.

R4 ← R1 OR R2

Step 8

Display all register values.

Step 9

Stop execution.

Register Transfer Operations

Operation 1 – Data Transfer

R3 ← R1

Before	**After**
R1 = 25	R1 = 25
R3 = 0	R3 = 25

Operation 2 – Addition

RTL:

R3 ← R1 + R2

Calculation:

25 + 15 = 40

Binary:

00011001

+

00001111

00101000

Result:

R3 = 40

Operation 3 – Subtraction

RTL:

R4 ← R1 − R2

Calculation

25 − 15 = 10

Binary

00011001

00001111

00001010

Result

R4 = 10

Operation 4 – AND Operation

RTL

R3 ← R1 AND R2

Binary

00011001

00001111

00001001

Decimal 9

Result

R3 = 9

Operation 5 – OR Operation

RTL

R4 ← R1 OR R2

Binary

00011001

00001111

00011111

Decimal 31

Result

R4 = 31

Simulation Table

Step	Operation	RTL Representation	Result
1	Load Data	R1=25, R2=15	Completed
2	Transfer	R3 ← R1	25
3	Addition	R3 ← R1 + R2	40
4	Subtraction	R4 ← R1 − R2	10
5	AND	R3 ← R1 AND R2	9
6	OR	R4 ← R1 OR R2	31

Sequence of Execution

Initialize Registers

Load Values into Registers

Transfer Data Between Registers

Addition

Subtraction

AND Operation

Role of the ALU

The Arithmetic Logic Unit plays a significant role in processor execution. During each instruction cycle, operands stored in registers are supplied to the ALU. Depending on the control signals generated by the control unit, the ALU performs the required arithmetic or logical operation and sends the result back to the destination register.

In this experiment:

- The ALU adds the values stored in R1 and R2.
- It subtracts the second operand from the first.
- It performs bitwise AND to identify common set bits.
- It performs bitwise OR to combine set bits from both operands.
- It updates the destination register after every operation.

Thus, the ALU serves as the computational engine of the processor, enabling efficient execution of instructions.

Observations

- Register transfer enables fast movement of data inside the CPU.
- Arithmetic operations are executed correctly by the ALU.
- Logical operations manipulate individual bits without changing register organization.
- Intermediate results are temporarily stored in destination registers.
- Register-based operations are significantly faster than memory-based operations because registers are located inside the processor.

Advantages

- High-speed data processing.
- Reduced memory access time.

Limitations

- Registers have limited storage capacity.
- Large datasets cannot be stored directly in registers.

Applications

- Microprocessors
- Embedded systems
- Digital Signal Processing (DSP)

Conclusion

The experiment successfully demonstrated the design and simulation of a register transfer system capable of performing arithmetic and logical operations using multiple registers and an Arithmetic Logic Unit (ALU). Data transfer between registers was verified using Register Transfer Language (RTL), and operations such as addition, subtraction, AND, and OR were executed accurately. The simulation showed that the ALU acts as the computational core of the processor by performing all arithmetic and logical computations while registers provide temporary storage for operands and results. This experiment provides a clear understanding of processor organization, data flow, and the importance of register transfer operations in achieving efficient instruction execution. It forms the foundation for advanced topics such as pipelining, hazard handling, and superscalar processor architecture.

Experiment No: 2

Title

Design and Simulation of Single-Bus and Multi-Bus Organization for Data Transfer Between Registers, Memory, and I/O Devices

Aim

To design and simulate a computer system using both single-bus and multi-bus organizations, perform data transfer operations among registers, memory, and input/output devices, compare their performance, and analyze how multiple bus structures improve processor efficiency by enabling parallel data transfer.

Objective

The objectives of this experiment are:

- To understand the architecture of single-bus and multi-bus organizations.
- To study the movement of data between CPU registers, memory, and I/O devices.
- To compare execution time and efficiency of both bus organizations.
- To analyze bus contention and bottlenecks.
- To understand the advantages of parallel data transfer in modern processors.

Theory

Bus Organization

A **bus** is a communication pathway that transfers data, addresses, and control signals between different components of a computer system. It acts as the backbone of data communication inside the processor and between the CPU, memory, and peripheral devices.

A typical bus consists of:

- **Data Bus** – Transfers data between components.
- **Address Bus** – Carries memory addresses.
- **Control Bus** – Carries control signals such as Read, Write, and Interrupt.

Efficient bus organization is essential because processor performance depends on how quickly data can be transferred among different hardware components.

Single-Bus Organization

In a **single-bus organization**, all processor registers, memory, and I/O devices share one common communication bus. Only one data transfer can occur at any given time.

Characteristics

- One common communication path.
- Only one transfer at a time.
- Low hardware complexity.
- Lower implementation cost.
- Increased waiting time due to bus contention.

Working Principle

Suppose Register R1 wants to send data to Register R2 while memory simultaneously wants to send data to the CPU. Since only one bus exists, one transfer must wait until the other completes.

Multi-Bus Organization

A **multi-bus organization** uses two or more buses to allow multiple transfers simultaneously.

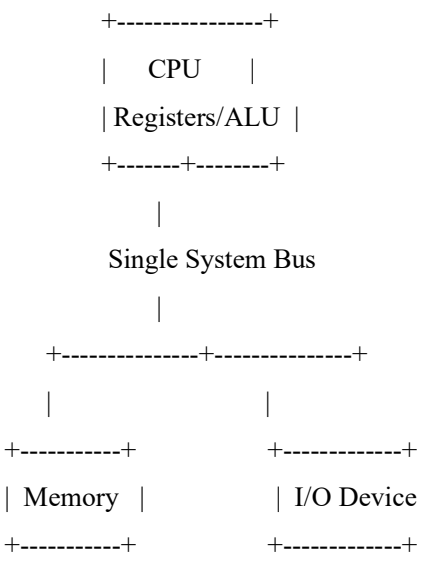
For example:

- Bus A transfers data from Register R1.
- Bus B transfers data from Register R2.
- Bus C writes results into Register R3.

Since multiple buses operate independently, several operations occur in parallel, improving processor performance.

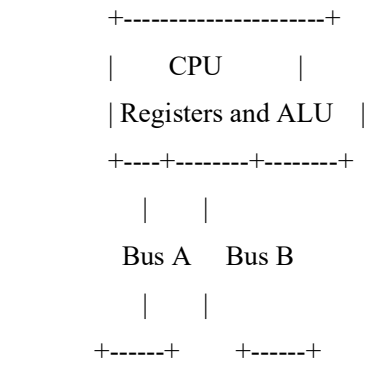
Block Diagram

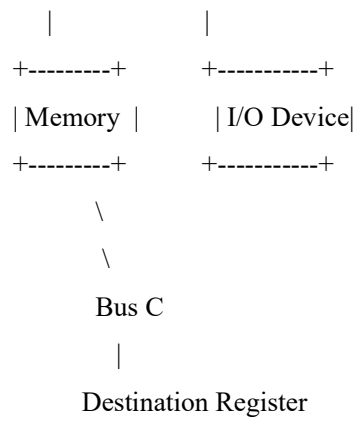
Single-Bus Organization



Only one communication path is available for all transfers.

Multi-Bus Organization





Multiple buses enable simultaneous communication between different components.

Components Used

- CPU Registers (R1, R2, R3)
- Arithmetic Logic Unit (ALU)
- Memory
- Input Device
- Output Device
- Single Bus
- Multi-Bus
- Control Unit

Algorithm

Step 1

Initialize registers, memory, and I/O devices.

Step 2

Design a single-bus architecture.

Step 3

Transfer data between registers.

Step 4

Transfer data between memory and CPU.

Step 5

Transfer data between CPU and I/O.

Step 6

Measure execution time.

Step 7

Design a multi-bus architecture.

Step 8

Repeat the same transfers using multiple buses.

Step 9

Measure execution time.

Step 10

Compare both architectures.

Step 11

Analyze efficiency.

Simulation

Assume the following initial values:

Component	Value
R1	20
R2	35
Memory Address M100	60
Input Device	15

Single-Bus Data Transfer

Operation 1

Transfer data from Memory to CPU Register.

Memory → R1

Operation 2

Transfer data from Input Device to Register.

Input → R2

Operation 3

Transfer Register to Memory.

R2 → Memory

Since only one bus exists, each operation must wait until the previous operation finishes.

Execution sequence:

Memory → CPU

↓

Input → CPU

↓

CPU → Memory

Total transfers = 3

Execution Time

Assume one transfer requires **5 ns**.

Total Time

3 × 5 = 15 ns

Multi-Bus Data Transfer

Now assume three buses are available.

At the same instant,

Bus A

Memory → R1

Bus B

Input → R2

Bus C

R2 → Memory

All three transfers occur simultaneously.

Execution sequence

Memory → CPU

||

Input → CPU

||

CPU → Memory

Execution Time

Only one transfer cycle is required.

5 ns

Performance Comparison

Parameter	**Single Bus**	**Multi Bus**
Number of Buses	1	3
Simultaneous Transfers	No	Yes
Bus Contention	High	Very Low
Parallel Execution	No	Yes
Data Throughput	Low	High
Hardware Cost	Low	High
Execution Time	15 ns	5 ns
Efficiency	Moderate	Excellent

Time Analysis

Single Bus

Transfer 1

0 – 5 ns

Transfer 2

5 –10 ns

Transfer 3

10 –15 ns

Total 15 ns

Multi Bus

All transfers

0 –5 ns

Total

5 ns

Speed Improvement

Speedup is calculated as:

Speedup = Time (Single Bus) / Time (Multi Bus)

Substituting the values:

Speedup = 15 / 5 = 3

Therefore, the multi-bus organization is three times faster than the single-bus organization for the given set of transfers.

Analysis

The simulation clearly demonstrates that a single-bus organization limits communication because all devices compete for the same bus. This causes **bus contention**, where one data transfer must wait until another completes, increasing execution time and reducing processor efficiency.

In contrast, a multi-bus organization allows independent buses to handle different transfers simultaneously. Registers, memory, and I/O devices can communicate in parallel, significantly reducing waiting time and improving overall throughput. Although multi-bus systems require more hardware and are costlier to implement, they provide much higher performance and are therefore widely used in modern processors.

Advantages of Single-Bus Organization

- Simple architecture.
- Easy to design and maintain.

Limitations of Single-Bus Organization

- Only one transfer at a time.
- High bus contention.

Advantages of Multi-Bus Organization

- Supports parallel data transfer.
- Reduces execution time.

Limitations of Multi-Bus Organization

- Higher hardware complexity.
- Increased implementation cost.

Applications

- Multi-core processors
- High-performance computing systems

The simulation of both bus organizations was successfully completed. The single-bus architecture executed data transfers sequentially, resulting in a total execution time of **15 ns** for three transfers. The multi-bus architecture executed the same transfers in parallel, completing them in **5 ns**, thereby achieving a **3× speedup**. This experiment confirms that multi-bus organization significantly enhances processor performance by enabling concurrent communication, reducing bottlenecks, and improving resource utilization.

Conclusion

This experiment successfully demonstrated the design and simulation of single-bus and multi-bus organizations for data transfer between registers, memory, and I/O devices. The comparative analysis showed that while a single-bus organization is simpler and more economical, it suffers from limited bandwidth and bus contention, leading to higher execution time. In contrast, the multi-bus organization enables simultaneous data transfers, significantly improving throughput and reducing communication delays. These observations highlight why modern processors employ multiple bus structures to support efficient parallel processing and high-speed data movement, making multi-bus organization a preferred choice for advanced computer architectures.

Experiment No: 3

Title

Design and Simulation of a Five-Stage Instruction Pipeline and Performance Evaluation

Aim

To design and simulate a **5-stage instruction pipeline** consisting of **Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB)** stages, execute a sequence of instructions, measure pipeline performance using **speedup** and **throughput**, and compare it with a non-pipelined processor.

Objective

The objectives of this experiment are:

- To understand the concept of instruction pipelining.
- To study the five stages of instruction execution.
- To simulate the execution of multiple instructions in a pipelined processor.
- To compare pipelined and non-pipelined execution.
- To calculate speedup and throughput.
- To analyze how pipelining improves processor performance.

Theory

Instruction Pipelining

Instruction pipelining is a processor design technique that divides instruction execution into several stages, allowing multiple instructions to be processed simultaneously. Similar to an assembly line in manufacturing, each stage performs a specific task, and different instructions occupy different stages at the same time.

Instead of waiting for one instruction to complete all stages before starting the next, pipelining overlaps the execution of instructions, significantly improving processor performance.

Although the execution time of an individual instruction does not decrease, the processor completes more instructions per unit time, increasing overall throughput.

Five Stages of the Instruction Pipeline

1. Instruction Fetch (IF)

- The processor fetches the instruction from memory.
- The Program Counter (PC) provides the address of the instruction.
- After fetching, the PC is updated to point to the next instruction.

2. Instruction Decode (ID)

- The fetched instruction is decoded.
- Source and destination registers are identified.
- The control unit generates the required control signals.

3. Execute (EX)

- The Arithmetic Logic Unit (ALU) performs arithmetic or logical operations.
- Effective memory addresses are calculated if required.

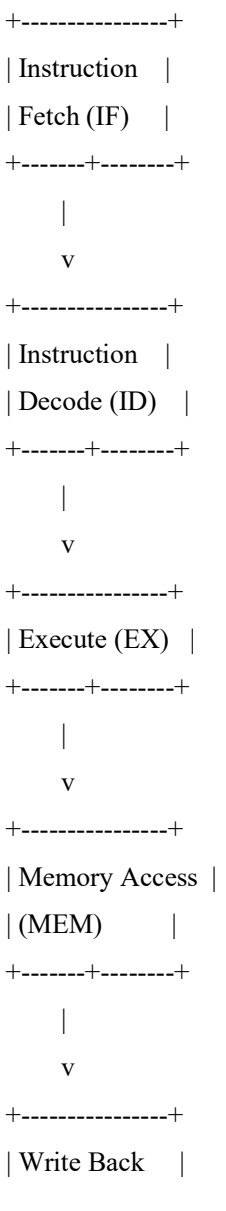
4. Memory Access (MEM)

- Data is read from memory (Load instruction).
- Data is written to memory (Store instruction).
- Arithmetic instructions usually bypass memory access.

5. Write Back (WB)

- The final result is written back into the destination register.
- The instruction execution is completed.

Block Diagram



| (WB) |

+----------+

Instructions Used for Simulation

Assume the following instructions:

Instruction	Operation
I1	R1 ← R2 + R3
I2	R4 ← R5 − R6
I3	R7 ← Memory[200]
I4	Memory[300] ← R8
I5	R9 ← R10 AND R11

Algorithm

Step 1

Initialize registers and memory.

Step 2

Fetch the first instruction.

Step 3

Decode the instruction.

Step 4

Execute the required ALU operation.

Step 5

Access memory if needed.

Step 6

Write the result into the destination register.

Step 7

Repeat the above steps for all instructions.

Step 8

Measure execution time.

Step 9

Calculate speedup and throughput.

Step 10

Compare with a non-pipelined processor.

Pipeline Execution Table

Clock Cycle	I1	I2	I3	I4	I5
1	IF				
2	ID	IF			
3	EX	ID	IF		

Clock Cycle	I1	I2	I3	I4	I5
4	MEM	EX	ID	IF	
5	WB	MEM	EX	ID	IF
6		WB	MEM	EX	ID
7			WB	MEM	EX
8				WB	MEM
9					WB

This table shows that after the pipeline is filled, multiple instructions are executed simultaneously in different stages.

Non-Pipelined Execution

Without pipelining, every instruction completes all five stages before the next instruction begins.

Instruction	Clock Cycles
I1	5
I2	5
I3	5
I4	5
I5	5

Total Clock Cycles

$5 \times 5 = 25$ cycles

Pipelined Execution

For a pipeline with **5 stages** executing **5 instructions**, the required clock cycles are:

Pipeline Cycles = Number of Instructions + Number of Stages − 1

Therefore,

Pipeline Cycles = $5 + 5 - 1 = 9$ cycles

Performance Calculation

Execution Time

Assume one clock cycle = **2 ns**

Non-Pipelined Processor

$25 \times 2 = 50$ ns

Pipelined Processor

$9 \times 2 = 18$ ns

Speedup

Formula

Speedup = Non-Pipelined Time / Pipelined Time

Calculation

Speedup = 50 / 18

Speedup ≈ 2.78

The pipelined processor is approximately **2.78 times faster** than the non-pipelined processor for the given instruction set.

Throughput

Throughput is defined as the number of instructions completed per unit time.

Non-Pipelined Processor

Throughput = 5 / 50

= 0.10 instructions/ns

Pipelined Processor

Throughput = 5 / 18

≈ 0.278 instructions/ns

The pipelined processor achieves nearly **2.8 times higher throughput** than the non-pipelined processor.

Performance Comparison

Parameter	**Non-Pipelined**	**Pipelined**
Number of Instructions	5	5
Pipeline Stages	5	5
Clock Cycles	25	9
Clock Period	2 ns	2 ns
Execution Time	50 ns	18 ns
Speedup	1	2.78
Throughput	0.10 instructions/ns	0.278 instructions/ns
Resource Utilization	Low	High
Processor Efficiency	Moderate	Excellent

Analysis

The simulation demonstrates that instruction pipelining significantly improves processor performance by overlapping the execution of multiple instructions. While the first instruction still requires five stages to complete, subsequent instructions enter the pipeline before previous instructions finish, ensuring continuous utilization of processor resources.

Compared to the non-pipelined processor, which executes instructions sequentially, the pipelined processor completes the same workload in only **9 clock cycles instead of 25**, reducing execution time by **64%**. The calculated speedup of approximately **2.78** and increased throughput highlight the effectiveness of pipelining in improving overall processor performance.

However, the ideal performance gain may be affected in practical systems due to hazards such as data dependencies, branch instructions, and resource conflicts, which introduce pipeline stalls. These challenges are addressed using advanced techniques such as forwarding, branch prediction, and hazard detection, which are explored in subsequent experiments.

Advantages

- Increases instruction throughput.
- Improves CPU utilization.

Limitations

- Pipeline hazards may reduce performance.
- Increased hardware complexity.

Applications

- Modern RISC processors (ARM, MIPS, RISC-V)
- Intel and AMD processors

Conclusion

The experiment successfully demonstrated the operation and performance benefits of a five-stage instruction pipeline. By allowing multiple instructions to occupy different stages simultaneously, the processor achieved better resource utilization, reduced execution time, and increased throughput compared to a non-pipelined architecture. The calculated speedup validates the effectiveness of pipelining as a fundamental performance enhancement technique in modern processors. This experiment also establishes a foundation for understanding more advanced topics such as pipeline hazards, forwarding mechanisms, branch prediction, superscalar execution, and out-of-order processing, which further improve processor performance in contemporary computer architectures.

Experiment No: 4

Title

Simulation and Analysis of Pipeline Hazards Using Stalling, Data Forwarding, and Branch Prediction

Aim

To simulate the occurrence of **data hazards, control hazards, and structural hazards** in a five-stage instruction pipeline, analyze their impact on processor performance, and implement hazard resolution techniques such as **stalling, data forwarding, and branch prediction** to improve execution efficiency.

Objective

The objectives of this experiment are:

- To understand different types of pipeline hazards.
- To simulate data, control, and structural hazards.
- To observe how hazards affect instruction execution.
- To implement stalling, forwarding, and branch prediction techniques.
- To compare pipeline performance before and after hazard resolution.
- To evaluate the effectiveness of hazard management techniques.

Theory

Pipeline Hazards

Instruction pipelining increases processor performance by executing multiple instructions simultaneously. However, when instructions interfere with one another, the normal flow of the pipeline is disrupted. Such situations are called **pipeline hazards**.

Pipeline hazards introduce delays known as **pipeline stalls** or **bubbles**, reducing processor efficiency.

There are three major types of hazards:

1. Data Hazard
2. Control Hazard
3. Structural Hazard

1. Data Hazard

A **data hazard** occurs when an instruction depends on the result of a previous instruction that has not yet completed

execution.

Example

I1 : R1 ← R2 + R3

I2 : R4 ← R1 + R5

Instruction **I2** requires the value of **R1**, but **I1** has not yet written the updated value into the register.

As a result, I2 cannot proceed immediately.

Pipeline without Hazard Resolution

Clock Cycle	I1	I2
1	IF	
2	ID	IF
3	EX	ID
4	MEM	STALL
5	WB	EX
6		MEM
7		WB

The processor inserts one stall cycle before I2 executes.

Effect of Data Hazard

- Processor waits for data.
- Pipeline efficiency decreases.
- Execution time increases.
- Throughput decreases.

Data Forwarding

Data forwarding (also called bypassing) transfers the ALU result directly from one pipeline stage to another without waiting for the Write-Back stage.

Instead of waiting,

EX → WB → Register → EX

the processor forwards the data directly:

EX → EX

Pipeline with Forwarding

Clock Cycle	I1	I2
1	IF	
2	ID	IF
3	EX	ID
4	MEM	EX
5	WB	MEM

Clock Cycle	I1	I2
6		WB

No stall cycle is required.

Advantages of Data Forwarding

- Eliminates unnecessary waiting.
- Improves throughput.
- Reduces execution time.
- Increases processor utilization.

2. Control Hazard

A **control hazard** occurs due to branch or jump instructions.

When the processor encounters a branch instruction, it does not immediately know which instruction should be fetched next.

Example

BEQ R1, R2, LABEL

The processor must first evaluate whether the branch condition is true.

During this period, the pipeline may fetch incorrect instructions.

Pipeline without Branch Prediction

Clock Cycle	Branch	Next Instruction
1	IF	
2	ID	IF
3	EX	STALL
4	MEM	STALL
5	WB	Correct Instruction

Two stall cycles are introduced.

Effect of Control Hazard

- Incorrect instruction fetch.
- Pipeline flush.
- Increased execution time.
- Reduced throughput.

Branch Prediction

Branch prediction allows the processor to predict whether the branch will be taken before the condition is fully evaluated.

Common prediction methods include:

- Static Branch Prediction
- Dynamic Branch Prediction

If the prediction is correct, instruction execution continues without interruption.

Pipeline with Branch Prediction

Clock Cycle	Branch	Predicted Instruction
1	IF	
2	ID	IF/
3	EX	ID
4	MEM	EX
5	WB	MEM
6		WB

The pipeline executes continuously without stalls.

Advantages of Branch Prediction

- Reduces branch penalty.

3. Structural Hazard

A **structural hazard** occurs when two or more instructions require the same hardware resource simultaneously.

Example

Suppose there is only **one memory unit**.

Instruction I1 needs memory access.

At the same time,

Instruction I2 attempts to fetch another instruction.

Both require memory simultaneously.

Only one can proceed.

Pipeline without Resolution

Clock Cycle	I1	I2
1	IF	
2	ID	IF
3	EX	ID
4	MEM	STALL
5	WB	EX
6		MEM
7		WB

A stall occurs because memory is busy.

Structural Hazard Resolution

Modern processors use:

- Separate Instruction Cache
- Separate Data Cache
- Multiple Memory Ports
- Multi-bus Architecture

These allow simultaneous instruction fetch and data access.

Pipeline after Structural Hazard Resolution

Clock Cycle	I1	I2
1	IF	
2	ID	IF
3	EX	ID
4	MEM	EX
5	WB	MEM
6		WB

No waiting occurs because hardware resources are available.

Simulation

Consider the following instructions:

I1 : R1 ← R2 + R3

I2 : R4 ← R1 + R5

I3 : BEQ R4, R6, LABEL

I4 : R7 ← Memory[200]

I5 : Memory[300] ← R8

Hazards observed:

Instruction	Hazard Type
I2 depends on I1	Data Hazard
I3 is Branch Instruction	Control Hazard
I4 and IF access memory simultaneously	Structural Hazard

Performance Analysis

Assume:

Pipeline without hazards

9 cycles

Pipeline with hazards

13 cycles

Pipeline after hazard resolution

10 cycles

Speedup after Hazard Resolution

Formula

Speedup = Time Before Resolution / Time After Resolution

Calculation

Speedup = 13 / 10 = 1.3

The hazard resolution techniques improve execution speed by approximately **30%** compared to the unresolved pipeline.

Comparison of Hazard Resolution Techniques

Hazard	Cause	Resolution Technique	Improvement
Data Hazard	Instruction dependency	Data Forwarding	Eliminates unnecessary stalls
Control Hazard	Branch instructions	Branch Prediction	Reduces branch penalty
Structural Hazard	Hardware resource conflict	Separate memory/cache, multi-bus	Allows parallel resource access

Overall Performance Comparison

Parameter	Without Resolution	After Resolution
Pipeline Stalls	High	Very Low
Execution Cycles	13	10
Processor Utilization	Low	High
Instruction Throughput	Moderate	High
Pipeline Efficiency	Reduced	Improved

Analysis

The simulation demonstrates that pipeline hazards significantly affect processor performance by introducing delays and reducing instruction throughput. Data hazards occur due to dependencies between instructions, control hazards arise from branch instructions, and structural hazards result from competition for hardware resources. These issues interrupt the smooth flow of the pipeline and decrease overall efficiency.

By implementing **data forwarding**, dependent instructions receive results directly from the ALU without waiting for register updates. **Branch prediction** minimizes delays caused by conditional branches by predicting the execution path in advance. **Improved hardware organization**, such as separate instruction and data memories or multi-bus architectures, eliminates structural conflicts. Together, these techniques reduce pipeline stalls, improve resource utilization, and significantly enhance processor performance.

Advantages

- Improves instruction throughput.
- Minimizes execution delays.

Limitations

- Hazard detection logic increases hardware complexity.
- Incorrect branch predictions require pipeline flushing.

Applications

- Modern Intel and AMD processors.
- ARM and RISC-V architectures.

Conclusion

This experiment successfully analyzed the impact of pipeline hazards on processor performance and demonstrated effective techniques to overcome them. Data forwarding eliminated unnecessary waiting caused by data dependencies, branch prediction reduced delays associated with control hazards, and improved hardware organization resolved structural conflicts. The comparative performance analysis confirmed that these hazard resolution techniques substantially enhance

pipeline efficiency by reducing stalls, improving throughput, and increasing processor utilization. These mechanisms are fundamental components of modern high-performance processors and are essential for achieving efficient instruction-level parallelism in contemporary computer architectures.

Experiment No: 5

Title

Analysis and Simulation of Superscalar Processor Architecture with Out-of-Order Execution, Speculative Execution, Hyper-Threading, and Simultaneous Multithreading (SMT)

Aim

To analyze and simulate the working of a **superscalar processor** that supports **out-of-order execution, speculative execution, hyper-threading, and simultaneous multithreading (SMT)**, and evaluate their impact on instruction throughput, processor utilization, and overall system performance.

Objective

The objectives of this experiment are:

- To understand the concept of superscalar processor architecture.
- To study out-of-order execution and speculative execution.
- To understand hyper-threading and simultaneous multithreading (SMT).
- To analyze parallel instruction execution.
- To compare processor performance with scalar processors.
- To evaluate throughput and resource utilization.

Theory

Superscalar Processor

A **superscalar processor** is an advanced CPU architecture capable of issuing and executing **multiple instructions during a single clock cycle** by using multiple execution units.

Unlike a scalar processor, which executes only one instruction per clock cycle, a superscalar processor identifies independent instructions and executes them simultaneously.

This improves processor performance without increasing the clock frequency.

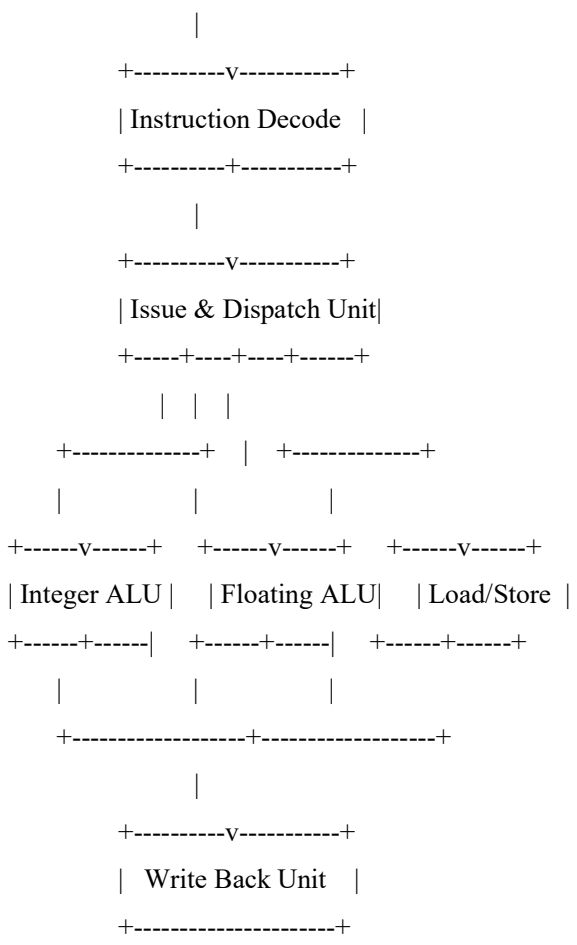
Working Principle

The processor contains multiple functional units such as:

- Integer ALUs
- Floating Point Units (FPU)
- Load/Store Units
- Branch Units

The instruction scheduler checks whether instructions are independent. If they are, they are dispatched to different execution units simultaneously.

Block Diagram



The dispatch unit assigns instructions to different execution units, enabling multiple instructions to execute concurrently.

Out-of-Order Execution

Definition

Out-of-order execution is a technique in which the processor executes instructions as soon as their operands become available, rather than strictly following the original program order.

Independent instructions are executed earlier while dependent instructions wait for the required data.

Example

I1 : R1 ← R2 + R3

I2 : R4 ← R1 + R5

I3 : R6 ← R7 + R8

Here:

- **I2** depends on the result of **I1**.
- **I3** is independent.

Instead of waiting:

I1 → I2 → I3

The processor executes:

I1 → I3 → I2

This keeps execution units busy and improves processor utilization.

Advantages of Out-of-Order Execution

- Reduces processor idle time.

- Improves instruction throughput.
- Increases hardware utilization.
- Enhances execution efficiency.

Speculative Execution

Definition

Speculative execution allows the processor to execute instructions before it is certain that they are needed.

The processor predicts the outcome of branch instructions using branch prediction.

If the prediction is correct, execution continues without delay.

If incorrect, speculative results are discarded, and the correct instruction path is executed.

Example

```
IF (A>B)
   Execute Block A
ELSE
   Execute Block B
```

The processor predicts the most likely branch and starts executing it before the branch condition is fully resolved.

Advantages of Speculative Execution

- Minimizes branch delay.
- Improves pipeline efficiency.

Hyper-Threading

Definition

Hyper-threading is Intel's implementation of simultaneous execution of two logical threads on a single physical processor core.

Each physical core appears as **two logical processors** to the operating system.

When one thread is waiting for data, another thread utilizes the idle execution resources.

Illustration

```
Physical Core
    |
+----+----+
| Thread 1 |
| Thread 2 |
+---------+
```

Both threads share the execution resources, resulting in better CPU utilization.

Advantages of Hyper-Threading

- Improves CPU utilization.
- Better multitasking performance.
- Increases throughput.
- Reduces idle execution units.

Simultaneous Multithreading (SMT)

Definition

Simultaneous Multithreading (SMT) is an advanced processor technology that allows instructions from multiple threads to

be executed simultaneously within the same clock cycle.

Unlike hyper-threading, which typically supports two threads per core, SMT can support multiple threads depending on the processor architecture.

Working

Example:

Thread 1 → ALU 1

Thread 2 → ALU 2

Thread 3 → Load Unit

Thread 4 → Floating Point Unit

Multiple execution units process instructions from different threads concurrently.

Difference Between Hyper-Threading and SMT

Parameter	**Hyper-Threading**	**SMT**
Developer	Intel	General processor architecture
Threads per Core	Usually 2	2 or more
Execution	Logical dual-threading	Multiple simultaneous threads
Hardware Utilization	High	Very High
Performance	Improved	Higher than Hyper-Threading

Simulation

Assume the processor executes the following instructions:

I1 : ADD R1,R2,R3

I2 : SUB R4,R5,R6

I3 : MUL R7,R8,R9

I4 : LOAD R10,[200]

I5 : STORE R11,[300]

Scalar Processor

Instructions executed:

Cycle 1 → I1

Cycle 2 → I2

Cycle 3 → I3

Cycle 4 → I4

Cycle 5 → I5

Execution Cycles

5 cycles

Superscalar Processor

Two instructions execute simultaneously.

Cycle 1 → I1 + I2

Cycle 2 → I3 + I4

Cycle 3 → I5

Execution Cycles

3 cycles

Performance Analysis

Assume one clock cycle = **2 ns**

Scalar Processor

Execution Time = 5 × 2= 10 ns

Superscalar Processor

Execution Time = 3 × 2= 6 ns

Speedup

Formula

Speedup = Scalar Time / Superscalar Time

Calculation

Speedup = 10 / 6≈ 1.67

Thus, the superscalar processor is approximately **1.67 times faster** than the scalar processor for the given instruction set.

Throughput

Scalar Processor

Throughput = 5 / 10= 0.50 instructions/ns

Superscalar Processor

Throughput = 5 / 6≈ 0.83 instructions/ns

The superscalar processor achieves significantly higher throughput due to parallel instruction execution.

Resource Utilization Comparison

Parameter	**Scalar Processor**	**Superscalar Processor**
Instructions per Cycle	1	Up to 2 or more
Execution Units Used	One	Multiple
Parallel Execution	No	Yes

Parameter	Scalar Processor	Superscalar Processor
Processor Utilization	Moderate	High
Throughput	Low	High
Execution Time	10 ns	6 ns
Overall Performance	Good	Excellent

Analysis

The simulation demonstrates that superscalar processors substantially improve performance by executing multiple independent instructions during the same clock cycle. Out-of-order execution prevents execution units from remaining idle by allowing independent instructions to proceed while dependent instructions wait for operands. Speculative execution minimizes branch delays by predicting likely execution paths, thereby maintaining a continuous instruction flow. Hyper-threading increases processor utilization by allowing two logical threads to share a physical core, while Simultaneous Multithreading (SMT) further enhances performance by enabling multiple threads to execute concurrently across available execution units.

The comparison with a scalar processor shows that the superscalar architecture completes the same workload in fewer clock cycles, increasing instruction throughput and making more effective use of hardware resources. These techniques are fundamental to the high performance achieved by modern multicore processors.

Advantages

- Executes multiple instructions simultaneously.
- Improves instruction throughput.

Limitations

- Increased hardware complexity.
- Higher power consumption.

Applications

- Intel Core and AMD Ryzen processors.
- ARM Cortex high-performance processors.

Conclusion

This experiment successfully demonstrated the architecture and operation of a superscalar processor along with advanced performance enhancement techniques, including out-of-order execution, speculative execution, hyper-threading, and simultaneous multithreading (SMT). The comparative analysis showed that these techniques enable multiple instructions and threads to execute concurrently, resulting in higher throughput, better resource utilization, and reduced execution time compared to a traditional scalar processor. The findings highlight that modern processors achieve superior computational efficiency not only through higher clock frequencies but also through intelligent instruction scheduling and parallel execution mechanisms, making superscalar architecture a cornerstone of contemporary high-performance computing systems.